Python

Method Overriding

Same method name in parent and child class, but different implementation!

Same Name Different Behaviour!

What is Method Overriding?

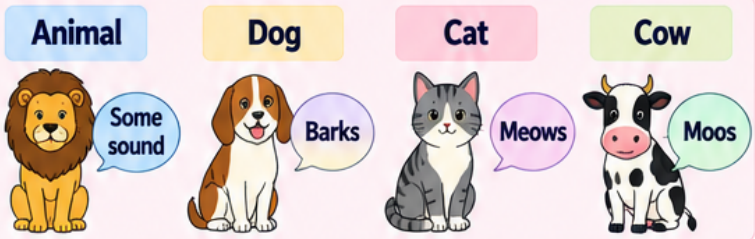
Method overriding means defining a method in the child class with the **same name** as a method in the parent class, but with a **different implementation.**

Key Points

- Occurs in inheritance (parent-child relationship).
- Same method name in parent and child class.
- Child class provides its own implementation.
- The method to be executed is decided at runtime based on the object.
- Helps in achieving polymorphism.
- Useful for customizing or extending the behavior of a method from the parent class.

Real-Life Example

A general Animal can make a sound. But a Dog, Cat, and Cow make different sounds even though they all have the same method name **sound()**.



Example: Method Overriding in Python

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")

class Cow(Animal):
    def sound(self):
        print("Cow moos")

# Creating objects
animals = [Animal(), Dog(), Cat(), Cow()]
for a in animals:
    a.sound()
```

Parent class method (sound)

Child class overrides the sound() method

Child class overrides the sound() method

Child class overrides the sound() method

The method to be executed is decided at runtime based on the object.

Output

```
Animal makes a sound
Dog barks
Cat meows
Cow moos
```

How It Works?

1. We create objects of different classes (Animal, Dog, Cat, Cow).
2. Each class has its own implementation of sound().
3. When we call a.sound(), Python runs the method of that specific object (runtime polymorphism).

Key Takeaway

Same method name, different behaviour based on the object!

"Method Overriding lets child classes give their own version of a method from the parent class."

Polymorphism in Action!

Python Generators

Generate values one at a time!

Don't store everything in memory. Generate when needed!

What are Generators?

A generator is a special type of function in Python that generates values **one at a time** instead of returning all values at once.

It uses the **yield** keyword instead of **return**.

Generators are memory efficient and useful when working with large data.

Key Points

- Uses **yield** instead of **return**.
- Generates values one at a time (on demand).
- Does not store all values in memory.
- Returns a generator object (we can iterate over it).
- Memory efficient (good for large data).
- Works like an iterator (supports next() and for loop).
- Useful for infinite sequences.

Syntax

```
def generator_function():
    yield value1
    yield value2
    yield value3
    ...
```

When the function is called, it returns a generator object. Values are produced one at a time when we iterate over it.

Example: Generator for Numbers

```
def count_numbers(n):
    for i in range(1, n + 1):
        yield i   # generates one value at a time

# Using the generator
gen = count_numbers(5)
print(gen)    # shows generator object
for num in gen:
    print(num)
```

Output

```
<generator object
count_numbers at 0x0000023A>
1
2
3
4
5
```

Real-Life Example



Think of a water tap. It gives water drop by drop when you need it, instead of filling the entire bucket at once.

This is how a generator works — it gives values **one at a time** when requested.

Generator vs Normal Function

Normal Function

```
def get_numbers(n):
    return [i for i in
            range(1, n + 1)]

nums = get_numbers(5)
print(nums)
```

- ✗ Returns all values at once (as a list).
- ✗ Stores all values in memory.
- ✗ Not efficient for large data.

Generator Function

```
def get_numbers(n):
    for i in range(1, n + 1):
        yield i

nums = get_numbers(5)
print(nums)
```

- ✓ Generates values one at a time.
- ✓ Does not store all values in memory.
- ✓ More memory efficient.

Common Use Cases

- Working with large files (read line by line).
- Processing large datasets.
- Infinite sequences (like natural numbers).
- Used in pipelines and data streaming.
- Saving memory in real-world applications.



Generators = Efficiency + On-Demand Values

"Generate when needed, not all at once!"

Python

Generators

Real-Life Examples

Don't store everything in memory. Generate when needed!

What are Generators?

A generator is a special type of function in Python that generates values **one at a time** instead of returning all values at once.

It uses the **yield** keyword (instead of **return**) and is **memory efficient**.

Key Points

- Uses **yield** instead of **return**.
- Generates values one at a time.
- Does not store all values in memory.
- Returns a generator object (iterator).
- Memory efficient (good for large data).
- Can be used in loops (`for`, `next()`).
- Useful for infinite sequences.

Syntax

```
def generator_function():
    yield value1
    yield value2
    yield value3
    ...
```

Each time we iterate, the function pauses and gives the next value (using **yield**).

Real-Life Examples of Generators

1 Water Tap



When you open a tap, water doesn't come all at once. It flows drop by drop, only when you need it.

Tap = Generator
Water drops = Values (one at a time)

2 YouTube Playlist



Videos are not downloaded all at once. The next video loads only when you watch it.

- Video 1
- Video 2
- Video 3
- Video 4

Playlist = Generator
Each video = Next value (on demand)

3 Online Food Orders

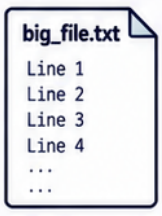


In a restaurant, orders are prepared one by one, not all at once. You get your food when it's ready.

- Order 1
- Order 2
- Order 3
- ...

Kitchen = Generator
Each dish = Next value (when ready)

4 Reading a Large File



A large file (like a log file) is read line by line instead of loading the entire file into memory.

File = Generator
Each `line` = Next `value` (one at a `time`)

5 Infinite Numbers

You can generate an infinite sequence (like natural numbers) without storing all numbers in memory.

1
2
3
4
5
...
...

∞

Sequence = Generator
Each number = Next value (on demand)

6 Sensor Data (Real World)

Sensors (like temperature sensors) generate data continuously. We read one value at a time as it is produced.



Sensor = Generator
Each reading = Next value (one at a time)



Generators help us work with large or infinite data in a simple and memory-efficient way.

Generate when needed, not all at once!

Source Code: YouTube Playlists (Using Generator)

Python

Real-World Example
Generate playlist videos one by one (as needed)!

This example shows how a generator can be used to get YouTube playlist videos one by one without loading all video details at once (memory efficient).

Python Source Code

```python
from googleapiclient.discovery import build

# --- Set your YouTube API Key ---
API_KEY = "YOUR_API_KEY_HERE"   # replace with your API key

# --- Function to generate videos from a playlist ---
def get_playlist_videos(playlist_id, max_results=5):
    youtube = build('youtube', 'v3', developerKey=API_KEY)
    next_page_token = None
    count = 0

    while True:
        request = youtube.playlistItems().list(
            part="snippet",
            playlistId=playlist_id,
            maxResults=max_results,
            pageToken=next_page_token
        )
        response = request.execute()

        for item in response['items']:
            title = item["snippet"]['title']
            video_id = item['snippet']['resourceId']['videoId']
            url = f"https://www.youtube.com/watch?v={video_id}"
            yield count + 1, title, url  # generator: gives one video at a time
            count += 1

        next_page_token = response.get('nextPageToken')
        if not next_page_token:
            break

# --- Example usage ---
if __name__ == "__main__":
playlist_id = "PLrAXtmRdnEQy8_tQKq7ZL9jz8h9KQ9KqF"  # replace with any playlist ID

print("🎬 YouTube Playlist Videos (Using Generator):\n")

for number, title, link in get_playlist_videos(playlist_id, max_results=5):
    print(f"{number}. {title}")
    print(f"   {link}\n")
```

Sample Output

```
🎬 YouTube Playlist Videos (Using Generator):

1. Python Full Course for Beginners
   https://www.youtube.com/watch?v=_uQrJ0TkZlc

2. Python Functions in Hindi
   https://www.youtube.com/watch?v=rfscVS0vtbw

3. Python Lists and Tuples
   https://www.youtube.com/watch?v=F6d9t_BuKLw

4. Python Dictionaries Explained
   https://www.youtube.com/watch?v=daefaLgNkw0

5. Python Projects for Beginners
   https://www.youtube.com/watch?v=8ext9G7xspg

... (more videos will be generated as you iterate)
```

How It Works?

1. We use YouTube Data API (v3) to get playlist items.
2. The function get_playlist_videos() is a generator (using `yield`).
3. It fetches a few videos at a time (max_results) from the playlist.
4. Each video (title and link) is yielded one by one.
5. We iterate over the generator using a for loop.
6. No need to store all videos in memory (good for large playlists).

Install Required Library

```
pip install google-api-python-client
```

Requirements

- Python 3.x
- google-api-python-client library
- YouTube Data API v3 key
- Internet connection
- Replace "YOUR_API_KEY_HERE" with your actual API key
- Replace playlist_id with any public playlist ID

Get YouTube API Key

1. Go to Google Cloud Console
2. Create a new project
3. Enable YouTube Data API v3
4. Create credentials (API Key)
5. Copy the API key and use it in the code

Generators help you handle large data efficiently

"One video at a time, when you need it!"

Stream Smart Not Heavy!

Python

Patient Record
(Using Generator)

Process one patient at a time!

Large hospital records? Don't load all at once. Use generators!



Sample Patient Records (patients.txt)

ID	Name	Age	Gender	Department
1001	Rahul Kumar	25	Male	OPD
1002	Priya Singh	30	Female	IPD
1003	Amit Verma	40	Male	Cardiology
1004	Neha Das	28	Female	Neurology
1005	Sohan Yadav	55	Male	Orthopedics
...	...	...	...	...

Python Code (Using Generator)

```
def read_patient_records(file_path):
    """Generator function to read patient records one by one"""
    with open(file_path, 'r') as file:
        for line in file:
            # skip empty lines
            if line.strip():
                parts = line.strip().split(','),
                # yield one patient record at a time
                yield {
                    'id': parts[0],
                    'name': parts[1],
                    'age': int(parts[2]),
                    'gender': parts[3],
                    'department": parts[4]
                }

# Example usage
for patient in read_patient_records('patients.txt'):
    print(f"ID: {patient['id']}, Name: {patient['name']}, "
          f"Age: {patient['age']}, Gender: {patient['gender']},"
          f"Department: {patient['department']}")
```

Output

```
ID: 1001, Name: Rahul Kumar, Age: 25, Gender: Male, Department: OPD
ID: 1002, Name: Priya Singh, Age: 30, Gender: Female, Department: IPD
ID: 1003, Name: Amit Verma, Age: 40, Gender: Male, Department: Cardiology
ID: 1004, Name: Neha Das, Age: 28, Gender: Female, Department: Neurology
ID: 1005, Name: Sohan Yadav, Age: 55, Gender: Male, Department: Orthopedics
...
```

How It Works?

1. The generator reads the file line by line.
2. Each line is processed and converted into a patient record (dictionary).
3. The record is **yielded** one at a time using **`yield`**.
4. No need to load all records into memory.
5. Useful for large hospital databases.

Generators make your code memory-efficient — perfect for real-world hospital systems!

Small Steps, Big Impact!

Python

File Handling

Work with files, store and read data!

Small Concepts Big Uses!

1 Opening a File

We use the **open()** function to open a file.

Syntax

```
file = open("filename", "mode")
```

Example

```
# Open a file in read mode
file = open("notes.txt", "r")
print("File opened successfully!")
```

Explanation

- "notes.txt" is the name of the file (with path if needed).
- "r" is the mode (read mode).
- The open() function returns a file object which can be used to perform operations like read(), write(), etc.

Common File Modes

Mode	Meaning
'r'	Open for reading (default)
'w'	Open for writing (overwrites file)
'a'	Open for appending (adds to file)
'x'	Create a new file (fails if exists)

2 Reading from a File

We can read data from a file using the **read()** method.

Example

```
# Read entire file
f = open("notes.txt", "r")
content = f.read()
print(content)
f.close()
```

Output (example)

```
Hello, Python!
File handling is easy.
Keep learning!
```

Explanation

- **read()** reads the entire content of the file as a single string.
- The content can be printed or processed as needed.
- Always close the file after use (using f.close()).



"Files store your data for the future!"

Learn Practice Apply!

Python

File Handling

More 2 Concepts

Manage files properly and write cleaner code!

Good File Handling = Better Programs!

5 Closing a File

We use the **close()** method to close a file after completing file operations.

Syntax

```
file.close()
```

Example

```
# Open, write and close a file
f = open("notes.txt", "w")
f.write("Hello, Python!")
f.close()
```

Explanation

- **close()** saves the changes and releases the file from memory.
- It is important to close the file after using it.
- If you don't close the file, it may remain open in memory (resource wastage).
- You can still use the file before closing it, but after **close()** you cannot perform any operation.

Note:
Always close the file using **close()** or use a **with** statement (recommended).

6 Using with Statement

The **with** statement automatically closes the file for us. We don't need to use **close()** manually.

Syntax

```
with open("filename", "mode") as f:
    # perform file operations
```

Example

```
# Read a file using with statement
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

Explanation

- The file is automatically closed when the block is finished, even if an error occurs.
- No need to use `f.close()`.
- It is cleaner, safer and recommended to use **with** statement.
- Helps in better resource management.

Equivalent Code (without with)

```
f = open("notes.txt", "r")
content = f.read()
print(content)
f.close()
```

Use Files Smartly, Code Confidently!

Small Concepts → Big Skills → Real Applications

Practice Implement Improve!

Python

Error Handling

Handle errors gracefully, keep your program running!

Errors Happen! Be Ready!

1 Try-Except Block

We use **try-except** to handle runtime errors so that the program does not crash.

Syntax

```
try:
    # code that may cause an error
except ExceptionType:
    # code to handle the error
```

Example

```
try:
    a = 10
    b = 0
    result = a / b
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Output

```
Cannot divide by zero!
```

Explanation

- The code inside **try** is executed.
- If an error occurs, control goes to **except**.
- The program does not stop, and a friendly message is shown instead of a crash.

2 Multiple Except Blocks

We can handle different types of errors using **multiple** except blocks.

Syntax

```
try:
    # code that may cause an error
except ErrorType1:
    # handle ErrorType1
except ErrorType2:
    # handle ErrorType2
except Exception:
    # handle any other error
```

Example

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
    print("Result:", result)
except ValueError:
    print("Please enter a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print("An unexpected error occurred:", e)
```

Output (Examples)

```
Input: abc
Please enter a valid number!

Input: 0
Cannot divide by zero!

Input: 5
Result: 20.0
```

Explanation

- We can use multiple **except** blocks for different errors.
- `ValueError` occurs when input is not a valid number.
- `ZeroDivisionError` occurs when dividing by zero.
- The last `except Exception` catches any other unexpected error.



Don't avoid errors. Handle them!

Good programs don't crash, they recover!

Learn Practice Build Better Programs!

Python

ValueError

Occurs when the value is not valid!

Right Value Matters!

What is ValueError?

- ValueError is raised when a function gets an argument of the **correct type**, but the **value is not valid**.
- It often occurs when we try to convert an invalid value to a different data type (like converting text to `int`).

Common Situations:

- Converting a string to int/float with invalid value
- Using invalid values in functions (like math.sqrt(-1))
- Invalid input in built-in functions

1 Example 1: Converting String to Integer

Code:

```
try:
    num = int("abc")
    print(num)
except ValueError:
    print("Invalid value! Please enter a number.")
```

Output:

```
Invalid value!
Please enter a number.
```

Explanation:

- "abc" is a string, but it cannot be converted to an integer.
- So, Python raises a **ValueError**.

2 Example 2: Invalid Value in Function

Code:

```
import math
try:
    result = math.sqrt(-1)
    print(result)
except ValueError:
    print("Cannot find square root of a
          negative number!")
```

Output:

```
Cannot find square root of a
negative number!
```

Explanation:

- The **math.sqrt()** function only works for non-negative numbers.
- Since -1 is not a valid value, it raises a **ValueError**.

How to Handle?

- Use **try-except** block to catch the **ValueError** and handle it gracefully.
- Give proper input or validate the value before using it.
- Show a user-friendly error message.

Key Takeaways

- **ValueError** occurs when the value is not valid.
- It happens even if the data type is correct.
- Use **try-except** to handle it.
- Always validate user input.

Handle Errors → Write Better Programs!

Learn Practice Improve!